eLSA — 딥테크 액셀러레이터 업무운영시스템

1. 프로젝트 정의

딥테크 액셀러레이터(창업기획자)의 **전주기 업무를 디지털화**하는 웹 기반 운영 플랫폼.
소싱 → 심사 → 투자 → 보육 → 수요기업 연결 → 후속투자 → 회수까지 **하나의 파이프라인**으로 연결한다.

핵심 설계 원칙

1. **릴레이 관리**: 팀 간 인계 흐름으로 관리 (전담이 아님)
2. **단일 기업 ID**: 소싱부터 사후관리까지 같은 ID + 같은 데이터룸
3. **문서·KPI 기반 의사결정**: 구두 아닌 기록과 수치로 결정
4. **성장 병목 제거**: 보육 = 이벤트가 아닌 개별 병목 제거 프로젝트
5. **PoC→계약→매출 전환**: 소개가 아닌 거래와 실증으로 연결
6. **백오피스 = 리스크 통제**: 행정지원이 아닌 투자·계약·컴플라이언스 통제 허브
7. **회수 전략은 투자 시점부터 설계**: 딥테크는 투자부터 exit 경로를 가정

대상 사용자

```
대표/파트너 (투자위원회 IC)
├── Sourcing팀: 딜발굴·파이프라인·1차스크리닝
├── 심사팀: 서류심사·인터뷰·DD·투자메모·IC상정
├── 보육팀: 온보딩·멘토링·KPI관리·IR/DemoDay
├── 오픈이노베이션팀: 수요기업매칭·PoC·실증·전략투자
└── 백오피스팀: 계약·집행·조합·보고·컴플라이언스
```

2. 마스터 스펙 참조

모든 구현은 반드시 아래 파일을 기반으로 해야 한다:

`CLAUDE_CODE_MASTER_PROMPT_v6.1_FINAL.md` (프로젝트 루트)

필요 정보	마스터 섹션
Enum 정의 (13종)	§3-2

필요 정보	마스터 섹션
테이블 설계 (37개 모델)	§3-3, §24, §25, §26, §31, §32
팀별 모듈 기능 명세	§4-1~4-5
인계 흐름 6가지	§5
RBAC 권한 매트릭스	§7
API 라우터 전체 목록	§9
프로젝트 디렉토리 구조	§10
구현 Phase 순서	§11
SOP 6개 워크플로우	§14
양식 14개 필드 스키마	§15
KPI 대시보드 (5팀)	§16
10개 핵심 자동화 로직	§18
KPI 자동 집계 수식	§19
Docker Compose	§20
시드 데이터	§21
.env 템플릿	§29
마이그레이션 순서	§30
기업 프로필 탭 구조	§28
Phase 1 퀵스타트	§35
Dockerfile	§41
DealStage 상태 머신	§44 (있으면)

3. 기술 스택

구분	기술	비고
Backend	**FastAPI** (Python 3.11+)	async, Pydantic v2
Frontend	**Next.js 14** (App Router)	TypeScript strict, Tailwind CSS, shadcn/ui
Database	**PostgreSQL 16**	UTF-8, 주 DB
Cache/Queue	**Redis 7**	캐시 + Celery broker
Task Queue	**Celery**	비동기 작업 (알림, KPI 집계, 보고서)
ORM	**SQLAlchemy 2.0**	async session
Auth	**JWT** + RBAC	팀별 권한 분리
Deploy	**Docker Compose**	로컬 + Cloudflare Tunnel
파일저장	로컬 volume (초기)	→ S3 호환 (확장 시)
Migration	**Alembic**	DB 스키마 버전 관리
Charts	**Recharts**	KPI 차트, 대시보드
DnD	**@hello-pangea/dnd**	칸반보드 드래그앤드롭
State	**zustand**	프론트엔드 상태 관리

4. 프로젝트 구조

```
elsa/
├── CLAUDE.md                          # 개발 가이드 (이 파일)
├── CLAUDE_CODE_MASTER_PROMPT_v6.1_FINAL.md  # 마스터 스펙
├── docker-compose.yml
├── .env / .env.example
│
├── backend/
│   ├── app/
│   │   ├── main.py                    # FastAPI 앱 진입점
│   │   ├── config.py                  # 설정 (Pydantic BaseSettings)
│   │   ├── database.py                # SQLAlchemy async session
│   │   ├── enums.py                   # 모든 Enum 정의 (중앙)
│   │   ├── errors.py                  # ErrorCode 패턴
│   │   ├── models/                    # SQLAlchemy 모델 (37개)
│   │   ├── schemas/                   # Pydantic v2 스키마
│   │   ├── routers/                   # API 라우터
│   │   ├── services/                  # 비즈니스 로직
│   │   ├── tasks/                     # Celery 비동기 작업
│   │   └── middleware/                # auth.py, rbac.py
│   ├── alembic/                       # DB 마이그레이션
│   ├── scripts/seed.py                # 시드 데이터
│   ├── requirements.txt
│   └── Dockerfile
│
├── frontend/
│   ├── src/
│   │   ├── app/                       # Next.js App Router
│   │   │   ├── (auth)/               # 인증 페이지
│   │   │   ├── sourcing/             # Sourcing팀 모듈
│   │   │   ├── review/               # 심사팀 모듈
│   │   │   ├── incubation/           # 보육팀 모듈
│   │   │   ├── oi/                   # 오픈이노베이션 모듈
│   │   │   ├── backoffice/           # 백오피스 모듈
│   │   │   ├── startup/[id]/         # 기업 통합 프로필 (13탭)
│   │   │   ├── admin/                # 관리 페이지
│   │   │   └── kpi/                  # KPI 대시보드
│   │   ├── components/
│   │   │   ├── ui/                    # shadcn/ui
│   │   │   ├── kanban/               # 칸반보드
│   │   │   ├── forms/                # 도메인 폼 컴포넌트
```

```
│   │   │   ├── charts/              # 차트 컴포넌트
│   │   │   └── layout/              # Sidebar, Header, AuthGuard
│   │   ├── lib/                      # api.ts, auth.ts, types.ts
│   │   └── hooks/                    # 커스텀 훅
│   ├── package.json
│   └── Dockerfile
│
└── nginx/nginx.conf
```

→ 전체 파일 목록은 마스터 §10 참조

5. Backend 컨벤션 (Python/FastAPI)

아키텍처 패턴: Router → Service → Model

```python
# routers/startups.py — HTTP 관심사만 처리
@router.post("/", response_model=StartupResponse)
async def create_startup(
    data: StartupCreate,
    current_user: User = Depends(get_current_user),
    db: AsyncSession = Depends(get_db),
):
    return await startup_service.create(db, data, current_user)

# services/startup_service.py — 비즈니스 로직
async def create(db: AsyncSession, data: StartupCreate, user: User) -> Startup:
    startup = Startup(**data.model_dump(), assigned_manager_id=user.id)
    db.add(startup)
    await db.flush()
    # DealFlow 자동 생성
    await pipeline_service.create_initial_flow(db, startup.id, user.id)
    # ActivityLog 기록
    await activity_log_service.log(db, user.id, startup.id, "create", {"entity": "startup"})
    await db.commit()
    await db.refresh(startup)
    return startup
```

타입 힌트

- Python 3.11+ 타입 힌트 **필수**
- Pydantic v2: `model_config = ConfigDict(from_attributes=True)`
- `Mapped[type]` 사용 (SQLAlchemy 2.0)

Enum

- 모든 Enum은 `enums.py` 에 중앙 관리 (마스터 §3-2)
- 새 Enum 추가 시 반드시 `enums.py` 에만 추가

에러 처리

```
from fastapi import HTTPException, status

raise HTTPException(
    status_code=status.HTTP_404_NOT_FOUND,
    detail="해당 스타트업을 찾을 수 없습니다."
)
```

- 에러 메시지는 **한글**로 작성
- 금액은 **Decimal** 타입 사용 (int/float 금지)
- 모든 timestamp는 **Asia/Seoul** (KST)

필수 규칙

- 삭제는 **soft delete**만 허용 (`is_deleted` 필드)
- 모든 변경사항은 **activity_logs**에 기록
- UUID를 Primary Key로 사용
- `async with AsyncSession()` 패턴 사용

6. Frontend 컨벤션 (TypeScript/Next.js)

기본 규칙

- App Router 사용 (`pages/` 디렉토리 아님)
- TypeScript **strict** mode

- shadcn/ui + Tailwind CSS
- 서버 컴포넌트 우선, 필요 시에만 `"use client"`
- API 호출은 `lib/api.ts` 로 중앙화 (axios)
- 한글 UI, 날짜 형식: `YYYY.MM.DD`

API 호출 패턴

```
// lib/api.ts
import axios from 'axios';

const api = axios.create({
  baseURL: process.env.NEXT_PUBLIC_API_URL,
});

api.interceptors.request.use((config) => {
  const token = localStorage.getItem('access_token');
  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});

export default api;
```

상태 관리

```
// hooks/useAuth.ts — zustand 사용
import { create } from 'zustand';

interface AuthState {
  user: User | null;
  token: string | null;
  login: (email: string, password: string) => Promise<void>;
  logout: () => void;
}
```

Backend Enum 미러링

- `lib/types.ts` 에 Backend Enum 값을 TypeScript로 미러링
- Enum 변경 시 양쪽 동시 업데이트 필수

파일 크기 제한

- 컴포넌트: 최대 400줄 (초과 시 분리)
- 함수: 최대 50줄

7. DB/모델 컨벤션

전체 모델 목록 (37개)

```
User, Startup, DealFlow, Screening, Review, InvestmentMemo,
ICDecision, InvestmentContract, CapTableEntry, Incubation,
MentoringSession, KPIRecord, PartnerDemand, PoCProject,
FollowOnInvestment, ExitRecord, HandoverDocument, Meeting,
Notification, ActivityLog, Report, SOPTemplate, SOPExecution,
FormTemplate, FormSubmission, TeamKPI, JobDescription,
DemoDay, InvestorMeeting, OrganizationSettings, Fund,
FundLP, FundInvestment, GovernmentProgram, Document,
Mentor, Batch
```

→ 각 모델의 필드 정의는 마스터 §3-3 ~ §32 참조

Base 모델 패턴

```python
import uuid
from datetime import datetime
from sqlalchemy import func
from sqlalchemy.orm import Mapped, mapped_column, DeclarativeBase

class Base(DeclarativeBase):
    pass

class BaseMixin:
    id: Mapped[uuid.UUID] = mapped_column(primary_key=True, default=uuid.uuid4)
    created_at: Mapped[datetime] = mapped_column(server_default=func.now())
    updated_at: Mapped[datetime] = mapped_column(server_default=func.now(), onupdate=func.now()
```

Soft Delete 패턴

```
is_deleted: Mapped[bool] = mapped_column(default=False)

# 조회 시 항상 필터
query = select(Startup).where(Startup.is_deleted == False)
```

테이블 네이밍

- 테이블명: snake_case 복수형 (예: `startups`, `deal_flows`)
- FK: `{entity}_id` (예: `startup_id`, `reviewer_id`)

Alembic 마이그레이션 순서 (FK 의존성)

→ 마스터 §30 참조. users → organization_settings → mentors → batches → startups → ... 순서대로 생성

8. API 설계 컨벤션

RESTful 라우터

- 기본 경로: `/api/v1/{resource}/`
- 전체 라우터 목록: 마스터 §9 참조

표준 CRUD 패턴

```
GET    /api/v1/startups/          # 목록 (페이지네이션)
GET    /api/v1/startups/{id}      # 상세
POST   /api/v1/startups/          # 생성
PUT    /api/v1/startups/{id}      # 전체 수정
PATCH  /api/v1/startups/{id}      # 부분 수정
DELETE /api/v1/startups/{id}      # Soft delete
```

페이지네이션

```
# 요청: ?page=1&page_size=20
# 응답:
{
    "data": [...],
    "total": 100,
    "page": 1,
    "page_size": 20,
    "message": "조회 성공"
}
```

필터링

- 쿼리 파라미터로 필터: `?industry=AI&stage=seed&is_portfolio=true`
- 검색: `?search=기업명`

에러 응답

```
{
    "detail": "해당 스타트업을 찾을 수 없습니다.",
    "error_code": "STARTUP_NOT_FOUND"
}
```

9. RBAC/인증

JWT 인증

- 알고리즘: HS256
- 토큰 만료: 24시간
- 리프레시 토큰: 별도 (확장 시)

역할/팀

```
# 역할 (role)
roles = ["partner", "analyst", "pm", "oi_manager", "backoffice", "admin"]

# 팀 (team)
teams = ["sourcing", "review", "incubation", "oi", "backoffice"]
```

권한 제한 규칙 (시스템 강제)

1. Sourcing팀은 최종 투자조건을 결정할 수 없다
2. 심사팀은 계약서 최종 보관/관리를 단독으로 하지 않는다
3. 보육팀은 투자 약속, 밸류에이션 협상을 하지 않는다
4. OI팀은 법적 구속력 있는 거래 약속을 단독으로 하지 않는다
5. 백오피스는 투자 판단, 성장전략 수립을 하지 않는다

→ 상세 권한 매트릭스: 마스터 §7

10. 비즈니스 로직

10대 핵심 자동화 (마스터 §18)

1. SRC-F01 제출 → Startup + DealFlow "inbound" 자동 생성
2. SRC-F02 A등급 + 인계=Y → Handover 자동 생성 + 심사팀 알림
3. INV-F01 10개 전체 수령 → DD 완료 상태 자동 전환
4. INV-F03 승인 → OPS-F01 + PRG-F01 동시 자동 생성
5. OPS-F01 10개 항목 완료 → Contract "closed" + Cap Table 자동 업데이트
6. PRG-F03 액션아이템 → Task 자동 등록 + 기한 알림
7. PRG-F04 KPI 3개월 연속 하락 → 위기 알림 + 등급 재검토
8. OI-F03 전환가능성 "높음" → 심사팀에 전략투자 검토 알림
9. OPS-F02 마감 7/3/당일 → 자동 리마인더
10. 모든 양식 제출 → ActivityLog에 자동 기록

DealStage 상태 머신

```
inbound → first_screening → deep_review → interview → due_diligence
→ ic_pending → ic_review → approved/conditional/on_hold/incubation_first/rejected
→ contract → closed → portfolio
```

- 단계 이동 시 DealFlow 자동 기록 + 해당 인계 문서 자동 생성

위기 감지 기준

- 현금고갈: runway 3개월 미만
- 핵심인력 이탈
- 고객 이탈
- 개발 지연
- 소송/규제 이슈
  → 감지 시 자동 알림 + 포트폴리오 등급 재검토

계약 클로징 조건

`pre_closing 7개 + 행정 10개 = 총 17개 체크 완료` (마스터 §36)

11. 환경/Docker

필수 환경변수

```
DB_USER, DB_PASSWORD, DB_NAME, DATABASE_URL
REDIS_URL
JWT_SECRET, JWT_ALGORITHM, JWT_EXPIRATION_HOURS
TZ=Asia/Seoul
UPLOAD_DIR, MAX_FILE_SIZE_MB
CELERY_BROKER_URL, CELERY_RESULT_BACKEND
NEXT_PUBLIC_API_URL
```

→ 전체 .env 템플릿: 마스터 §29

Docker Compose 서비스

```
db (PostgreSQL 16) | redis (Redis 7) | backend (FastAPI)
celery_worker | celery_beat | frontend (Next.js)
```

→ 상세 구성: 마스터 §20

Celery Beat 스케줄

작업	주기
KPI 자동 집계	매월 1일 02:00
보고 마감 리마인더	매일 09:00
인계 미확인 에스컬레이션	매시간
위기 신호 스캔	매일 08:00

12. 구현 Phase

Phase	이름	핵심 산출물
1	기반 인프라 + 인증 + Startup CRUD	Docker, 스키마, JWT, RBAC, 스타트업 CRUD
2	Sourcing팀 모듈	딜플로우 칸반, 스크리닝 폼, 인계 패키지
2.5	양식 엔진	FormTemplate CRUD, DynamicFormRenderer
3	심사팀 모듈	서류심사 5축, 인터뷰, DD, 투자메모, IC
3.5	심사팀 양식	INV-F01~F03 + 자동 트리거
4	백오피스팀 모듈	계약 워크플로우, Cap Table, 보고센터
5	보육팀 모듈	포트폴리오, 온보딩, 멘토링, KPI
5.5	보육팀 양식	PRG-F01~F04 + 자동 트리거
6	오픈이노베이션팀 모듈	파트너 수요맵, PoC, 정부사업
7	팀간 연결 + 고도화	인계 허브, 회의체, Celery 태스크, 통합 대시보드

Phase	이름	핵심 산출물
8	KPI 대시보드 + 전사 뷰	팀별 KPI 5종, 전사 경영 대시보드
9	SOP 엔진 + JD	SOP 워크플로우, JD 관리

13. 현재 Phase 상태

Phase	상태	최종 업데이트
Phase 1	🔄 진행 중	2026-03-16
Phase 2~9	⏳ 대기	-

Phase 1 세부 진행

- [x] CLAUDE.md 생성
- [ ] Docker 인프라 + 프로젝트 스켈레톤
- [ ] DB 스키마 + Enum + 핵심 모델
- [ ] 인증 + RBAC + 사용자 API
- [ ] Startup CRUD + 프론트엔드

14. 테스트 전략

Backend

- **pytest** + **httpx** (async API 테스트)
- **pytest-asyncio** (async 테스트)
- 테스트 DB: 별도 PostgreSQL DB 또는 테스트 시작 시 생성/삭제
- 최소 커버리지: **80%**
- 테스트 파일: `tests/test_{module}.py`

Frontend

- Phase 1에서는 수동 테스트 중심

- Phase 5 이후 E2E 테스트 추가 검토 (Playwright)

15. 커밋 컨벤션

형식

```
[모듈] 설명

예시:
[infra] Docker Compose + Dockerfile 설정
[auth] JWT 인증 + RBAC 미들웨어 구현
[startup] 스타트업 CRUD API 구현
[sourcing] 딜플로우 칸반보드 구현
```

모듈 태그

```
infra, auth, startup, sourcing, review, incubation, oi,
backoffice, form, sop, kpi, common
```

주석

- 주석은 **한글**로 작성
- 불필요한 주석 금지 (코드가 자체 설명적이어야 함)
- 복잡한 비즈니스 로직에만 주석 추가

배포 환경

- 로컬 개발: `C:\Users\jghyu\elsa`
- 외부 접근: Cloudflare Tunnel (`jghyun.cloudflareaccess.com`)
- 다중 사용자 동시 접속 지원 필수